

06_deployment_metrics

December 2, 2025

0.1 1. Deployment, Inference and Operational Metrics.

This notebook evaluates the behaviour of the deployed chest X-ray inference service when executed on CPU and GPU through containerised FastAPI endpoints running inside a local Kubernetes (Minikube) cluster. The focus is on characterising latency and basic throughput under realistic single-image request patterns, using the same preprocessing configuration and class mappings as the training notebooks. ##### Rationale: - Connect the previous training and evaluation workflow (Notebooks 01–05) with a concrete, running inference service.

- Ensure that the deployed API (CPU and GPU) exposes a consistent interface and returns stable predictions.

- Quantify per-image latency on CPU and GPU, using identical inputs and preprocessing, to support an operational comparison.

- Export metrics (e.g. latency summaries) to disk so they can be incorporated directly into the Thesis Results and Discussion chapters.

- Document a reproducible procedure for starting the cluster, forwarding ports and collecting measurements from the deployed service.

```
[3]: # 2. Define base paths and inference endpoints.
# Import standard libraries.
from pathlib import Path

# Define project root. Resolve the notebook directory and move to project root.
# Always resolve paths explicitly to maintain reproducibility across machines.
project_root = Path.cwd().resolve().parents[0]

# Define the CPU and GPU inference endpoints exposed via port-forward.
# The notebook must communicate with the running Kubernetes services.
CPU_URL = "http://localhost:8000"
GPU_URL = "http://localhost:8001"

# Print confirmation for quick sanity-check.
print("Project root:", project_root)
print("CPU endpoint:", CPU_URL)
print("GPU endpoint:", GPU_URL)
```

Project root: /lab/px/msc-sept-24-ft-cohort-capstone-chrisanich

CPU endpoint: http://localhost:8000

GPU endpoint: http://localhost:8001

0.2 2. Health-Check of Inference Endpoints.

Before collecting operational metrics, confirm that the CPU and GPU inference services running inside the Kubernetes cluster are reachable through the active port-forwarding tunnels. This ensures that the notebook interacts with live deployments and avoids taking measurements on unresponsive or misconfigured services. ##### Rationale: - Verify that both deployments are running and accepting requests.

- Detect connectivity or deployment errors before benchmarking.
- Ensure consistent behaviour across all measurements.
- Maintain reproducibility of the full deployment workflow.

```
[4]: # 3. Health-check for CPU and GPU inference services
import requests

def check_health(url: str):
    """
    Send a GET request to /health and return the JSON response.
    Keep the function minimal to avoid masking deployment errors.
    """
    try:
        r = requests.get(f"{url}/health", timeout=5)
        r.raise_for_status()      # Force an exception on non-200 responses.
        return r.json()
    except Exception as e:
        return {"error": str(e)}

# Execute health checks for both endpoints.
cpu_health = check_health(CPU_URL)
gpu_health = check_health(GPU_URL)

print("CPU health:", cpu_health)
print("GPU health:", gpu_health)
```

```
CPU health: {'status': 'ok'}
GPU health: {'status': 'ok'}
```

0.3 3. Load Preprocessing Configuration.

Operational metrics must use the same preprocessing pipeline that was applied during model training and evaluation. The deployed API already uses this configuration internally, but the notebook also needs to load it so that image preparation, class interpretation and result validation remain consistent with the earlier notebooks. ##### Rationale - Ensure preprocessing in the notebook matches the deployed service.

- Maintain consistent image size, normalisation statistics and class mappings.
- Prevent discrepancies between notebook-generated inputs and API behaviour.
- Support accurate interpretation of predictions returned by the CPU and GPU endpoints.

```
[5]: # 4. Load preprocessing configuration.
```

```

import json

# Define the path to the shared preprocessing configuration.
# This file ensures that image size, normalisation statistics,
# and class mappings are identical to those used during training.
config_path = project_root / "experiments" / "pools" / "preprocessing_config.
↪json"

# Load the configuration. Keep this step explicit to avoid
# accidental differences between notebooks and the deployed API.
with open(config_path, "r") as f:
    cfg = json.load(f)

# Extract commonly used fields for later use.
img_size = int(cfg["img_size"])
class_to_index = cfg["class_to_index"]

print("Loaded preprocessing configuration.")
print("Image size:", img_size)
print("Classes:", class_to_index)

```

```

Loaded preprocessing configuration.
Image size: 256
Classes: {'Pneumonia': 0, 'TB': 1, 'Normal': 2}

```

0.4 4. Image Loading and Preprocessing Helper.

To benchmark inference latency consistently, the notebook must prepare images using the same transformations applied during training. Although the deployed API applies the correct preprocessing internally, the notebook needs a local helper function for preparing image data before sending requests. This ensures that inputs used for timing and stress-testing behave identically across CPU and GPU endpoints. ##### Rationale: - Maintain strict alignment between notebook preprocessing and the API pipeline.

- Guarantee that latency measurements reflect the true operational behaviour of the deployed models.
- Avoid accidental performance differences caused by inconsistent image handling.
- Provide a reusable helper for single-image and batch experiments.

```

[6]: # 5. Define image loading and preprocessing helper

from PIL import Image
import torch
from torchvision import transforms

# Construct the preprocessing pipeline. Use the same operations
# as during training to maintain consistency with the deployed API.
# This includes resizing, centre cropping, tensor conversion,
# and normalisation using the loaded configuration.

```

```

preprocess = transforms.Compose([
    transforms.Resize(img_size),
    transforms.CenterCrop(img_size),
    transforms.ToTensor(),
    transforms.Normalize(cfg["imagenet_mean"], cfg["imagenet_std"])
])

def load_and_prepare_image(path: Path):
    """
    Load an image from disk and apply the standard preprocessing pipeline.
    Keep the function minimal and explicit to avoid any divergence from
    the training workflow.
    """
    img = Image.open(path).convert("RGB")
    x = preprocess(img).unsqueeze(0) # Add batch dimension.
    return x

```

Insights:

- No errors while loading the image from disk and applying the standard preprocessing pipeline.

0.5 5. Single-Image Latency Benchmark.

With preprocessing configured and health checks completed, the notebook now measures the time required for the CPU and GPU inference services to process a single image. This benchmark provides a direct comparison between both deployments under identical conditions, using the same input and preprocessing pipeline. *### Rationale:* - Quantify end-to-end latency for a single inference request on CPU and GPU.

- Provide a controlled measurement that reflects realistic clinical usage.
- Establish a baseline for later throughput and stress-testing experiments.
- Support inclusion of operational results in the thesis with reproducible timing data.

[]: *# 5. Single-image latency measurement for CPU and GPU.*

```

import time
import requests

def measure_latency(url: str, img_path: Path, n_iterations: int = 5):
    """
    Measure end-to-end latency by sending the same image to the
    specified inference endpoint multiple times. Keep the timing
    loop minimal to avoid overhead unrelated to the API call.
    """
    latencies = []

    # Read the image bytes once to avoid file I/O affecting timing.
    with open(img_path, "rb") as f:
        img_bytes = f.read()

```

```

for _ in range(n_iterations):
    start = time.perf_counter()

    response = requests.post(
        f"{url}/predict",
        files={"file": ("image.png", img_bytes)},
        timeout=10
    )

    end = time.perf_counter()
    latencies.append(end - start)

return latencies, response.json()

# Select a test image for benchmarking.
# Modify this path if needed.
test_image = project_root / "data" / "tbx11k" / "imgs" / "tb" / "tb0003.png"

# Execute CPU and GPU latency measurements.
cpu_latencies, cpu_pred = measure_latency(CPU_URL, test_image)
gpu_latencies, gpu_pred = measure_latency(GPU_URL, test_image)

print("CPU latencies (s):", cpu_latencies)
print("GPU latencies (s):", gpu_latencies)
print("CPU prediction:", cpu_pred)
print("GPU prediction:", gpu_pred)

```

```

CPU latencies (s): [0.11013478800123266, 0.13261667500046315,
0.09931859899916162, 0.10842507000052137, 0.09468968200053496]
GPU latencies (s): [0.15547900899946399, 0.17507769800067763,
0.020912656000291463, 0.01572499200119637, 0.18052895399887348]
CPU prediction: {'prediction_index': 1, 'prediction_class': 'TB',
'architecture': 'victorio', 'domain': 'cxr', 'device': 'cpu', 'class_map':
{'Pneumonia': 0, 'TB': 1, 'Normal': 2}}
GPU prediction: {'prediction_index': 1, 'prediction_class': 'TB',
'architecture': 'victorio', 'domain': 'cxr', 'device': 'cuda', 'class_map':
{'Pneumonia': 0, 'TB': 1, 'Normal': 2}}

```

0.6 7. Summary Statistics for Single-Image Latency.

Raw latency measurements for individual requests are useful for debugging, but they are difficult to interpret directly. To compare CPU and GPU behaviour in a concise way, the notebook now computes standard summary statistics (mean and standard deviation) for the single-image latency measurements collected from each endpoint. ##### Rationale: - Convert raw latency traces into interpretable summary values.

- Enable a direct numerical comparison between CPU and GPU performance.
- Provide statistics that can be quoted in the Results and Discussion chapters.
- Prepare the data for later export to CSV or JSON tables for reproducibility.

```
[8]: # 6. Compute summary statistics for CPU and GPU latency measurements.
import numpy as np

def summarize(latencies):
    """
    Return mean and standard deviation for a list of float latencies.
    Keep the output minimal so it can be reused in tables and plots.
    """
    arr = np.array(latencies, dtype=float)
    return {
        "mean": float(arr.mean()),
        "std": float(arr.std()),
        "n": len(arr),
    }

cpu_stats = summarize(cpu_latencies)
gpu_stats = summarize(gpu_latencies)

print("CPU latency statistics:", cpu_stats)
print("GPU latency statistics:", gpu_stats)
```

```
CPU latency statistics: {'mean': 0.10903696280038275, 'std':
0.013098652768367669, 'n': 5}
GPU latency statistics: {'mean': 0.10954466180010058, 'std':
0.07496809417149591, 'n': 5}
```

0.6.1 7.1. Summary of CPU vs GPU Inference Latency:

Rationale:

- Provides a compact statistical overview of latency behaviour across repeated calls.
- Helps identify stability, variance, and potential warm-up effects in GPU execution.
- Ensures both inference services behave consistently before enabling full metrics collection.

Overview:

- The table below summarises mean latency, standard deviation, and sample size for both CPU and GPU inference paths.
- CPU shows lower variance and stable behaviour.
- GPU has similar mean latency but noticeably higher variance due to first-call warm-up and CUDA context initialisation.

Both services return identical predictions, confirming consistency in the deployment.

0.7 7. Aggregate Metrics Table and Export.

To make the operational results useful for the written thesis, the CPU and GPU latency statistics must be stored in a structured, version-controlled format. This section organises the summary values into a small table and exports them to disk so that they can be reused in the Results chapter and any later visualisation without rerunning the notebook. ##### Rationale: - Convert CPU and GPU latency statistics into a compact, tabular representation.

- Store metrics alongside the rest of the project outputs under **reports/tables**.
- Ensure that operational findings are reproducible and can be cited in the thesis.
- Allow later plotting or aggregation without repeating the full benchmark.

```
[9]: # 7. Build aggregate latency table and export to disk
import pandas as pd
from datetime import datetime

# Define output directory under reports/tables.
# Keep the path explicit to remain consistent with other notebooks.
tables_dir = project_root / "reports" / "tables"
tables_dir.mkdir(parents=True, exist_ok=True)

# Build a small DataFrame with CPU and GPU latency statistics.
# Use simple column names so the table is easy to reuse in the thesis.
rows = [
    {
        "backend": "cpu",
        "device_reported": cpu_pred.get("device", "cpu"),
        "architecture": cpu_pred.get("architecture", "unknown"),
        "domain": cpu_pred.get("domain", "unknown"),
        "mean_latency_s": cpu_stats["mean"],
        "std_latency_s": cpu_stats["std"],
        "n_samples": cpu_stats["n"],
    },
    {
        "backend": "gpu",
        "device_reported": gpu_pred.get("device", "cuda"),
        "architecture": gpu_pred.get("architecture", "unknown"),
        "domain": gpu_pred.get("domain", "unknown"),
        "mean_latency_s": gpu_stats["mean"],
        "std_latency_s": gpu_stats["std"],
        "n_samples": gpu_stats["n"],
    },
]

metrics_df = pd.DataFrame(rows)
metrics_df["timestamp"] = datetime.utcnow().isoformat(timespec="seconds")

# Define output file paths.
csv_path = tables_dir / "k8s_operational_metrics.csv"
```

```

json_path = tables_dir / "k8s_operational_metrics.json"

# Export metrics in both CSV and JSON formats.
metrics_df.to_csv(csv_path, index=False)
metrics_df.to_json(json_path, orient="records", indent=2)

print("Saved metrics to:")
print(" -", csv_path)
print(" -", json_path)
metrics_df

```

Saved metrics to:

```

- /lab/px/msc-sept-24-ft-cohort-capstone-
chrisanich/reports/tables/k8s_operational_metrics.csv
- /lab/px/msc-sept-24-ft-cohort-capstone-
chrisanich/reports/tables/k8s_operational_metrics.json

```

```

[9]:  backend device_reported architecture domain  mean_latency_s  std_latency_s  \
0      cpu              cpu      victorio      cxr          0.109037      0.013099
1      gpu              cuda      victorio      cxr          0.109545      0.074968

      n_samples      timestamp
0           5  2025-11-22T14:20:30
1           5  2025-11-22T14:20:30

```

0.8 Conclusion:

- A dedicated deployment experiment was conducted to characterise the operational behaviour of the containerised inference service under CPU and GPU execution within the local Kubernetes environment.
- For a fixed chest X ray image drawn from the TBX11K test set, five repeated requests were issued to each FastAPI endpoint via port forwarding, and the end to end latency of each request was measured.
- For the victorio architecture fine tuned on the CXR domain, the CPU service exhibited a mean latency of approximately 0.109 s with a standard deviation of 0.013 s across five runs, while the GPU service reported a very similar mean latency of approximately 0.110 s but with a larger standard deviation of 0.075 s, reflecting the impact of CUDA initialisation and warm up effects on early requests.
- In all cases the predicted class and index were identical across CPU and GPU for the test image, confirming functional equivalence of the deployed models and consistency of the pre-processing and checkpoint loading pipeline across devices.
- The resulting metrics, including backend type, reported device, architecture, domain, mean latency, standard deviation, sample size and timestamp, were exported to a structured table under `reports/tables/k8s_operational_metrics` for subsequent inclusion in the Results and Discussion chapters.